Smart CCTV Violence detection system

A project report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering
(Artificial Intelligence and Data Science)

By

Vayigandla Tulasi [22B01A45B8]
Ravuri Yamini [22B01A4595]
Vetsa Yasaswi [22B01A45C3]
Sajja Tejaswini Surya Sindhu [22B01A45A0]

Under the supervision of

Dr. G. Durga Prasad
Asst.Professor

Department Of Artificial Intelligence
Shri Vishnu Engineering College for Women (Autonomous)
Bhimavaram-534 202, India
April, 2026
2025-26

SHRI VISHNU ENGINEERING COLLEGE FOR WOMEN
(AUTONOMOUS)
VISHNUPUR, BHIMAVARAM-534 202
DEPARTMENT OF ARTIFICIAL INTELLIGENCE

DECLARATION

I hereby declare that this project report entitled "**Smart CCTV Violence detection system**" submitted by me for the award of the degree of **Bachelor of Technology in Computer Science and Engineering(Artificial Intelligence and Data Science)** in the Department of Artificial Intelligence, Shri Vishnu Engineering College for Women (Autonomous), Bhimavaram , India, is a record of original project work carried out by me. This work has not been submitted either in part or whole to any other university or institution, in India or abroad, for the award of any degree.

Name	**Regd.No.**	**Signature**
Vayigandla Tulasi	22B01A45B8	
Ravuri Yamini	22B01A4595	
Vetsa Yasaswi	22B01A45C3	
Sajja Tejaswini Surya Sindhu	22B01A45A0	

SHRI VISHNU ENGINEERING COLLEGE FOR WOMEN
(AUTONOMOUS)
VISHNUPUR, BHIMAVARAM-534 202
DEPARTMENT OF ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the Project Report entitled "**Smart CCTV Violence detection system**" that is being submitted by **Vayigandla Tulasi [22B01A45B8]**, **Ravuri Yamini [22B01A4595]**, **Vetsa Yasaswi [22B01A45C3]**, **Sajja Tejaswini Surya Sindhu [22B01A45A0]** in partial fulfillment for the award of **Bachelor of Technology** in **Computer Science and Engineering(Artificial Intelligence and Data Science)** to the Department of Artificial Intelligence, Shri Vishnu Engineering College for Women(Autonomous), Bhimavaram is a record of bonafide work did by them under my supervision during the academic year 2025 - 26. To the best of my insight, the work consolidated in this postulation has not been submitted somewhere else for the honor of any degree.

Internal Guide
Dr.G.Durga Prasad
Asst.Professor
Department of AI
SVECW, Bhimavaram

Dr.A.Sri Krishna
Head of the Department
Department of AI
SVECW, Bhimavaram

External Examiner

ACKNOWLEDGEMENT

Behind every achievement lies an unfathomable sea of gratitude to those who activated it, without whom it would never ever come into existence. To them we lay a words of gratitude imprinted within us.

We wish to place our deep sense of gratitude to **Sri. K. V. Vishnu Raju**, Chairman of SVECW for his constant support on our each and every progressive work.

We are thankful to **Dr. G. Srinivasa Rao**, Principal of SVECW, for being a source of inspiration and constant encouragement.

We wish to express our sincere thanks to **Dr. P. Venkata Rama Raju**, Vice-Principal of SVECW, for being a source of inspiration and constant encouragement.

We wish to express our sincere thanks to **Dr. A. Sri Krishna**, Head of the Department, Artificial Intelligence, for being a source of inspiration and constant encouragement.

We are highly obliged in taking the opportunity to sincerely thank our Guide **Dr. G. Durga Prasad**, for his support and constant encouragement throughout the project. We are indebted for his guidance that proved to be very much helpful in completing our project successfully in time.

PROJECT ASSOCIATES

S.No	Name	Regd.No.
1)	Vayigandla Tulasi	22B01A45B8
2)	Ravuri Yamini	22B01A4595
3)	Vetsa Yasaswi	22B01A45C3
4)	Sajja Tejaswini Surya Sindhu	22B01A45A0

ABSTRACT

Closed-Circuit Television (CCTV) systems are widely deployed in public and private spaces to enhance safety and security. However, continuous manual monitoring of surveillance footage is impractical due to the large number of cameras and the sustained attention required from human operators. Consequently, violent incidents may go undetected or be identified only after they occur, limiting the effectiveness of surveillance systems. Automated violence detection using computer vision and deep learning techniques has emerged as a promising solution to address this challenge.

This project presents an intelligent violence detection system for CCTV surveillance videos based on a hybrid deep learning architecture. The proposed system combines a lightweight convolutional neural network, **MobileNetV2**, for spatial feature extraction with a Long Short-Term Memory (LSTM) network for temporal sequence modeling. By analyzing sequences of video frames rather than individual images, the system captures both appearance and motion patterns associated with violent activities. This spatiotemporal approach improves the ability to distinguish violent behavior from visually similar non-violent actions.

The model is trained and evaluated using the **RWF-2000 dataset**, which contains real-world surveillance videos with diverse scenes, lighting conditions, and camera viewpoints. Preprocessing steps include frame extraction, resizing, normalization, and sequence generation to prepare video data for model training. The proposed framework is designed to be computationally efficient, making it suitable for deployment in resource-constrained surveillance environments.

Experimental results demonstrate that the system can effectively classify video sequences into violent and non-violent categories, showing promising performance on realistic CCTV footage. The proposed approach highlights the potential of deep learning-based spatiotemporal models to assist security personnel by enabling automated detection of violent incidents and supporting proactive surveillance systems.

ABBREVIATIONS

AI	Artificial Intelligence
CCTV	Closed-Circuit Television
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
ConvLSTM	Convolutional Long Short-Term Memory
DL	Deep Learning
CV	Computer Vision
RGB	Red, Green, Blue
FPS	Frames Per Second
GPU	Graphics Processing Unit
IoU	Intersection over Union
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative

Table of Contents

DECLARATION **i**

CERTIFICATE **ii**

ACKNOWLEDGEMENT **iii**

ABSTRACT **iv**

Abbreviations **v**

Table of Contents **vi**

List of Tables **xi**

List of Figures **xii**

List of Algorithms **xiii**

1 Introduction **1**

1.1 Background of Artificial Intelligence in Surveillance Systems 1

1.2 Video-Based Action Recognition 2

1.3 Violence Detection in Surveillance Videos 3

1.4 Challenges in Automated Violence Detection 4

1.4.1 Temporal Understanding 4

1.4.2 Computational Constraints 5

1.4.3 Environmental Variability 5

1.4.4 Class Imbalance and Ambiguity 5
1.5 Problem Statement 5
1.6 Objectives 6
1.7 Motivation 6
1.8 Scope of the System 7
1.9 Contributions of the Project 7
1.10 Organization of Project Report 7

2 Literature Review **9**
2.1 Artificial Intelligence in Video Surveillance 9
2.2 Video-Based Action Recognition 10
2.3 Spatiotemporal Deep Learning Models 11
2.3.1 3D Convolutional Neural Networks 11
2.3.2 CNN–RNN Architectures 11
2.3.3 Convolutional LSTM Networks 12
2.3.4 Transformer-Based Video Models 12
2.4 Violence Detection in Surveillance Videos 12
2.5 Real-Time Requirements and Deployment Considerations 13
2.6 Surveillance Datasets for Violence Detection 13
2.7 Lightweight Deep Learning Architectures 14
2.8 Evaluation Metrics for Violence Detection 14
2.9 Research Gaps 14
2.10 Summary 15

3 Methodology **16**
3.1 System Overview 16
3.2 System Architecture 17
3.3 Dataset Description 17
3.3.1 Dataset Statistics 18
3.4 Dataset Preprocessing 19

3.4.1 Frame Extraction 19
3.4.2 Frame Resizing 19
3.4.3 Normalization 19
3.4.4 Sequence Formation 19
3.4.5 Training Configuration 19
3.5 Video Processing and Frame Extraction 20
3.6 Image Preprocessing 20
3.6.1 Frame Resizing 20
3.6.2 Normalization 20
3.6.3 Sequence Formation 20
3.7 Spatial Feature Extraction Using MobileNetV2 21
3.8 Temporal Modeling Using LSTM 21
3.9 Classification Layer 22
3.10 Training Procedure 22
3.11 Operational Workflow 22
3.12 Algorithms 23
3.13 Summary 23

4 Implementation **24**
4.1 Development Environment 24
4.2 Software Tools and Libraries 25
4.2.1 TensorFlow and Keras 25
4.2.2 OpenCV 25
4.2.3 NumPy 25
4.2.4 Matplotlib 26
4.3 Dataset Preparation 26
4.4 Video Processing 26
4.4.1 Frame Extraction 26
4.4.2 Frame Sampling Strategy 27

4.5 Image Preprocessing 27
4.5.1 Resizing 27
4.5.2 Normalization 28
4.5.3 Sequence Formation 28
4.5.4 Frame Sampling Strategy 29
4.5.5 Preparation of Model Input 30
4.6 Feature Extraction Using MobileNetV2 30
4.7 Temporal Modeling Using LSTM 30
4.8 Classification Layer 31
4.9 Training Process 31
4.10 Testing and Prediction 32
4.11 Performance Evaluation 32
4.12 Execution Workflow 32
4.13 Implementation Challenges 33
4.14 Summary 33

5 Experimental Setup and Results 34
5.1 Experimental Setup 34
5.2 Performance Evaluation 35
5.2.1 Classification Accuracy 36
5.2.2 Confusion Matrix Analysis 36
5.2.3 ROC Curve Evaluation 36
5.2.4 Training and Validation Behavior 37
5.2.5 Qualitative Evaluation 37
5.3 Summary 37

6 Result Analysis and Discussion 38
6.1 Implementation Details 38
6.2 Dataset and Evaluation Protocol 38
6.3 Training Configuration 39

6.4 Classification Performance 39
6.5 Quantitative Performance Metrics 40
6.6 ROC Curve Analysis 40
6.7 Training and Convergence Behavior 41
6.8 Class-wise Performance Analysis 42
6.9 Inference Performance 43
6.10 Comparison with Baseline Methods 43
6.11 Robustness Evaluation 43
6.12 Qualitative Results 44
6.13 Real-Time System Demonstration 44
6.14 User Interface Output 44
6.15 Administrative Dashboard 45
6.16 Automated Email Alert System 46
6.17 System Performance in Real-Time Deployment 48
6.18 Alert Logging and Monitoring 48

7 Conclusions and Future Work 49

7.1 Conclusions 49
7.2 Future Work 50

A Code Attachments 52

A.1 Frame Sequence Preparation 52
A.2 Dataset Loader and Frame Normalization 53
A.3 MobileNetV2 Feature Extraction 53
A.4 LSTM Temporal Classification Model 54
A.5 Real-Time Violence Detection Pipeline 54
A.6 Alert Generation Module 55

References 56

List of Tables

6.1 Performance Metrics of the Proposed Model 40
6.2 Comparison with Baseline Methods 43

List of Figures

3.1 Proposed CNN–LSTM Violence Detection Architecture 17
3.2 Dataset distribution of RWF-2000 18
3.3 Sample frames from dataset 18

6.1 Confusion matrix obtained on the test set. 40
6.2 ROC curve illustrating the trade-off between true positive rate and false positive rate. 41
6.3 Training and validation loss across epochs. 41
6.4 Training and validation accuracy across epochs. 42
6.5 Qualitative results. Left: violent activity detected. Right: non-violent scenario correctly classified. 44
6.6 User interface showing real-time violence detection with confidence score 45
6.7 Admin dashboard displaying alert logs and system statistics 46
6.8 Automated email alert with detection details and location information 47

List of Algorithms

3.1 Violence Detection Pipeline 23

3.2 Sequence Classification 23

CHAPTER 1

Introduction

1.1 Background of Artificial Intelligence in Surveillance Systems

Artificial Intelligence (AI) has become a transformative technology across numerous domains, including healthcare, transportation, finance, manufacturing, and security. In recent years, AI-driven computer vision systems have significantly enhanced automated monitoring and decision-making capabilities in surveillance applications. With the rapid increase in urbanization and public infrastructure, the deployment of Closed-Circuit Television (CCTV) cameras has expanded dramatically to ensure safety in public spaces such as airports, railway stations, shopping malls, educational institutions, and residential areas.

Traditional surveillance systems rely heavily on human operators to continuously monitor video feeds. However, the large number of cameras and the prolonged attention required make manual monitoring inefficient and error-prone. Human operators may miss critical events due to fatigue, distraction, or cognitive overload. Consequently, important incidents such as violent altercations, assaults, or abnormal crowd behavior may go unnoticed or be detected only after significant delay. These limitations have motivated the development of intelligent surveillance systems capable of automatically analyzing video streams and detecting suspicious activities.

Advances in deep learning and computer vision have enabled machines to in-

terpret visual data with remarkable accuracy. Convolutional Neural Networks (CNNs) have proven particularly effective in extracting spatial features from images, while Recurrent Neural Networks (RNNs), especially Long Short-Term Memory (LSTM) networks, are capable of modeling temporal dependencies in sequential data [6]. By combining these techniques, modern AI systems can analyze video sequences to recognize complex human activities and behaviors.

Violence detection in surveillance footage represents a critical application of AI-based video analysis. Automated detection systems can assist security personnel by providing real-time alerts, enabling rapid response to dangerous situations, and improving overall public safety. Survey studies highlight the growing importance of intelligent surveillance technologies in modern security infrastructures [10, 9].

1.2 Video-Based Action Recognition

Violence detection is closely related to the broader field of video-based action recognition, which focuses on identifying human activities from video data. Unlike image classification, video analysis requires understanding both spatial appearance and temporal motion patterns. Actions such as running, fighting, or falling unfold over multiple frames, making temporal modeling essential.

Early approaches to action recognition relied on handcrafted spatiotemporal features such as Space-Time Interest Points (STIP) [8] and motion descriptors derived from optical flow [4]. These methods captured motion information by analyzing changes between consecutive frames. Although effective in controlled environments, handcrafted features often struggle with real-world surveillance footage due to variations in lighting, viewpoint, background clutter, and occlusion.

To address these limitations, deep learning approaches have been introduced. Two-stream convolutional neural networks process spatial information from RGB frames and motion information from optical flow separately [13]. While this approach significantly improved recognition accuracy, the computation of optical

flow is resource-intensive, limiting real-time deployment.

Three-dimensional convolutional neural networks (3D CNNs) extend standard 2D convolutions into the temporal dimension, enabling direct learning of spatiotemporal features from video sequences [14]. Although effective, 3D CNNs require substantial computational resources and large datasets for training.

An alternative approach combines 2D CNNs with recurrent networks such as LSTM to capture temporal dependencies across frame sequences. CNN–LSTM architectures have demonstrated strong performance in various video analysis tasks, including violence detection [15]. This hybrid approach balances accuracy and computational efficiency, making it suitable for practical surveillance applications.

More recently, transformer-based models such as Video Vision Transformers (ViViT) have been proposed to model long-range temporal dependencies [1]. However, these models typically require extensive computational resources and large-scale training data.

1.3 Violence Detection in Surveillance Videos

Violence detection aims to automatically identify aggressive or harmful activities such as physical fights, assaults, or abnormal crowd behavior in video footage. Early research focused on detecting violent crowd dynamics using motion patterns [5]. Modern approaches leverage deep learning to analyze both visual appearance and motion cues.

Real-world surveillance footage presents several challenges:

- Variations in lighting conditions and weather
- Different camera angles and resolutions
- Occlusion caused by objects or crowds
- Background clutter and dynamic environments

- Low video quality or compression artifacts

These factors make violence detection significantly more complex than action recognition in controlled datasets. Therefore, robust models must generalize well across diverse environments.

The availability of realistic datasets has accelerated research in this field. The RWF-2000 dataset provides a large collection of real-world surveillance videos containing violent and non-violent scenarios [3]. Unlike staged datasets, RWF-2000 captures authentic CCTV footage with diverse scenes, making it suitable for evaluating practical surveillance systems.

Other datasets exist for violence detection, but many are domain-specific or limited in scale [2]. Consequently, developing systems that perform reliably on real-world data remains an active research challenge.

1.4 Challenges in Automated Violence Detection

Despite significant progress, automated violence detection systems face several technical challenges.

1.4.1 Temporal Understanding

Violent activities evolve over time and cannot be reliably identified from a single frame. For example, two individuals standing close together may appear similar to a fight in a static image. Temporal context is necessary to distinguish between normal interactions and aggressive behavior. Models must therefore capture motion patterns across sequences of frames.

1.4.2 Computational Constraints

Surveillance systems often operate continuously and must process large volumes of video data in real time. High-complexity models may achieve high accuracy but are impractical for deployment due to hardware limitations. Efficient architectures such as MobileNet are designed to reduce computational cost while maintaining performance [7, 11].

1.4.3 Environmental Variability

CCTV cameras operate in diverse environments, including indoor and outdoor settings, crowded spaces, and poorly illuminated areas. A robust system must generalize across these conditions without requiring environment-specific tuning.

1.4.4 Class Imbalance and Ambiguity

Violent events are relatively rare compared to normal activities. This imbalance makes training reliable classifiers challenging. Additionally, some activities may be ambiguous, requiring contextual interpretation.

1.5 Problem Statement

Although CCTV systems are widely deployed, their effectiveness is limited by the reliance on manual monitoring. Human operators cannot continuously analyze multiple video feeds with consistent attention, leading to delayed detection of critical incidents.

The problem addressed in this project is the development of an automated system capable of analyzing surveillance video sequences and accurately classifying them as violent or non-violent. The system must capture both spatial and temporal characteristics of human behavior while remaining computationally efficient for practical deployment.

1.6 Objectives

The primary objective of this project is to design and implement an automated violence detection system for CCTV surveillance videos using deep learning techniques.

The specific objectives are as follows:

- To extract spatial features from individual video frames using a lightweight convolutional neural network.
- To model temporal dependencies across frame sequences using a recurrent neural network.
- To classify video sequences into violent and non-violent categories based on learned spatiotemporal representations.
- To evaluate the performance of the proposed model on a real-world surveillance dataset.
- To design a computationally efficient system suitable for practical deployment.

1.7 Motivation

The motivation for this project arises from the increasing need for intelligent security systems capable of proactive threat detection. Rapid urbanization and population growth have led to increased demand for public safety measures. Automated violence detection can assist law enforcement agencies, security personnel, and emergency responders by providing early warnings of dangerous situations.

By leveraging advances in deep learning, it is possible to develop systems that continuously monitor surveillance footage without human fatigue. Such systems can improve response times, reduce crime impact, and enhance overall safety.

1.8 Scope of the System

The proposed system focuses on binary classification of surveillance videos into violent and non-violent categories. It processes video sequences extracted from CCTV footage and analyzes both spatial and temporal features.

The system is designed as a decision-support tool for surveillance monitoring. It does not identify individuals, perform facial recognition, or predict future behavior. Instead, it detects ongoing violent activities to assist human operators.

1.9 Contributions of the Project

The key contributions of this project include:

- Development of a hybrid CNN–LSTM framework for violence detection in surveillance videos.
- Utilization of MobileNetV2 as an efficient feature extractor suitable for resource-constrained environments.
- Implementation of sequence-based classification to capture temporal context.
- Evaluation on the RWF-2000 dataset to demonstrate effectiveness on real-world CCTV footage.

1.10 Organization of Project Report

The organization of the project report is as follows.

Chapter 1 Introduction – Presents the background, problem statement, objectives, motivation, and scope of the project.

Chapter 2 Literature Review – Discusses existing research related to action recognition and violence detection.

Chapter 3 Methodology – Describes the proposed system architecture and data processing pipeline.

Chapter 4 Implementation – Details the implementation of the model and software components.

Chapter 5 Experimental Setup and Results – Presents experimental procedures and evaluation results.

Chapter 6 Result Analysis and Discussion – Provides detailed analysis of performance and observations.

Chapter 7 Conclusion and Future Work – Summarizes findings and suggests future improvements.

CHAPTER 2

Literature Review

2.1 Artificial Intelligence in Video Surveillance

Artificial Intelligence (AI) has transformed modern surveillance systems by enabling automated interpretation of visual data. Traditional surveillance infrastructures relied primarily on human operators to monitor video feeds continuously. However, the rapid expansion of CCTV networks in urban environments has made manual monitoring increasingly impractical. Large public spaces such as transportation hubs, commercial complexes, and educational institutions may deploy hundreds or thousands of cameras simultaneously, generating enormous volumes of video data.

AI-based surveillance systems address this challenge by automatically analyzing video streams and detecting abnormal or suspicious activities. Computer vision algorithms can identify objects, track motion, recognize behaviors, and generate alerts in real time. Such systems reduce the burden on human operators and improve response times during critical incidents.

Recent studies emphasize that intelligent surveillance technologies play a vital role in modern security frameworks, enabling proactive threat detection rather than reactive investigation [10]. These systems are increasingly used for crime prevention, crowd monitoring, traffic management, and emergency response coordination. Despite these advantages, designing robust AI models that perform reliably under diverse environmental conditions remains a major research chal-

lenge.

2.2 Video-Based Action Recognition

Action recognition is a core task in computer vision that involves identifying human activities from video sequences. Unlike image classification, which analyzes static visual content, video analysis must capture temporal dynamics. Human actions such as walking, running, fighting, or falling unfold over time and require sequential modeling.

Early action recognition research focused on handcrafted spatiotemporal features. Space-Time Interest Points (STIP) detect regions in video where motion changes significantly, enabling the identification of dynamic events [8]. Optical flow methods estimate pixel-level motion between consecutive frames, providing information about direction and speed of movement [4]. These techniques were widely used due to their interpretability and relatively low computational requirements.

However, handcrafted features have limited ability to generalize across real-world conditions. Surveillance footage often contains noise, low resolution, camera shake, and complex backgrounds, which reduce the effectiveness of manually engineered descriptors. Additionally, designing features that capture all relevant aspects of human behavior is extremely difficult.

To overcome these limitations, deep learning methods have become dominant. Two-stream convolutional neural networks process spatial information from RGB frames and temporal information from optical flow separately [13]. This architecture demonstrated significant performance improvements but introduced additional computational cost due to optical flow estimation.

2.3 Spatiotemporal Deep Learning Models

Deep learning approaches enable automatic learning of features directly from raw video data. These models can capture complex spatial and temporal relationships without manual feature engineering.

2.3.1 3D Convolutional Neural Networks

Three-dimensional convolutional neural networks extend conventional 2D convolutions into the temporal dimension. By applying convolution operations across consecutive frames, 3D CNNs can learn motion patterns and temporal dependencies simultaneously [14]. This approach eliminates the need for explicit motion computation.

Despite their effectiveness, 3D CNNs require substantial computational resources and memory. Training such models often demands large datasets and high-performance hardware, making them less suitable for real-time surveillance deployment.

2.3.2 CNN–RNN Architectures

An alternative approach combines convolutional neural networks with recurrent neural networks. CNNs extract spatial features from each frame, while recurrent models analyze temporal sequences of these features.

Long Short-Term Memory (LSTM) networks are particularly well suited for sequential data because they can retain information over long time intervals [6]. CNN–LSTM architectures have been successfully applied to video classification tasks, including violence detection, activity recognition, and anomaly detection [15]. This hybrid architecture provides a balance between accuracy and computational efficiency.

2.3.3 Convolutional LSTM Networks

Convolutional LSTM (ConvLSTM) networks extend traditional LSTM by incorporating convolution operations within the recurrent structure. This design preserves spatial information while modeling temporal dependencies [12]. ConvLSTM models are particularly useful for spatiotemporal prediction tasks.

2.3.4 Transformer-Based Video Models

Recent advances in deep learning have introduced transformer architectures for video understanding. Video Vision Transformers (ViViT) use self-attention mechanisms to model long-range dependencies across frames [1]. While these models achieve strong performance, they require extensive computational resources and large training datasets, limiting their practicality for surveillance applications.

2.4 Violence Detection in Surveillance Videos

Violence detection focuses on identifying aggressive or harmful activities such as physical fights, assaults, or abnormal crowd behavior. Early research analyzed motion irregularities to detect violent interactions in crowds [5]. These methods relied heavily on handcrafted features and were sensitive to environmental variations.

Modern deep learning approaches analyze both appearance and motion cues to recognize violent behavior. CNN-based models can capture contextual information such as body posture and interaction patterns, while temporal models detect dynamic movements associated with aggression.

Real-world surveillance footage presents numerous challenges, including low resolution, poor lighting, occlusion, and complex backgrounds. Violence detection systems must operate reliably under these conditions while maintaining low false alarm rates. Survey studies highlight the importance of robust models capable of

generalizing across diverse environments [9, 10].

2.5 Real-Time Requirements and Deployment Considerations

In practical surveillance scenarios, violence detection systems must operate in real time. Delayed detection reduces the usefulness of automated alerts, as security personnel may not be able to respond promptly.

Real-time processing requires models that balance accuracy with computational efficiency. High-complexity architectures may achieve superior performance in laboratory settings but are impractical for deployment on edge devices or embedded systems. Latency, memory usage, and power consumption are critical factors.

Additionally, surveillance systems must operate continuously for extended periods. This requirement necessitates stable and efficient algorithms capable of processing large volumes of data without performance degradation.

2.6 Surveillance Datasets for Violence Detection

Reliable datasets are essential for training and evaluating violence detection models. Many early datasets consisted of staged scenarios or domain-specific videos, limiting their applicability to real-world surveillance.

The RWF-2000 dataset provides a large collection of real-world CCTV videos containing both violent and non-violent events [3]. The dataset includes diverse scenes, camera viewpoints, and lighting conditions, making it suitable for evaluating practical systems.

Other datasets exist but are often limited in scale or diversity [2]. The scarcity of large, realistic datasets remains a major obstacle in developing highly robust models.

2.7 Lightweight Deep Learning Architectures

Efficiency is a key requirement for surveillance systems. MobileNet architectures were developed to reduce computational cost while maintaining competitive accuracy. By using depthwise separable convolutions, MobileNet significantly reduces the number of parameters compared to traditional CNNs [7].

MobileNetV2 introduced inverted residual structures and linear bottlenecks, further improving efficiency and performance [11]. Lightweight models enable deployment on resource-constrained devices and support real-time inference.

2.8 Evaluation Metrics for Violence Detection

Evaluating the performance of violence detection systems requires appropriate metrics. Common measures include accuracy, precision, recall, F1-score, and confusion matrices. In surveillance applications, minimizing false negatives is particularly important, as missed violent events may have serious consequences.

Robust evaluation also requires testing models under diverse conditions to assess generalization capability.

2.9 Research Gaps

Despite substantial progress, several challenges remain in automated violence detection. Many high-performance models are computationally expensive and unsuitable for real-time deployment. Additionally, generalization across different environments remains difficult due to variability in scene characteristics.

Another limitation is the lack of integrated approaches that effectively combine spatial feature extraction, temporal modeling, and computational efficiency within a single framework.

2.10 Summary

This chapter reviewed the existing research on artificial intelligence in video surveillance, action recognition techniques, spatiotemporal deep learning models, violence detection methods, and lightweight architectures. Traditional approaches relied on handcrafted features, whereas modern systems employ deep learning models to learn complex patterns directly from video data.

Although significant advances have been achieved, challenges related to real-time processing, environmental variability, and dataset limitations remain. The proposed project addresses these issues by developing an efficient CNN–LSTM based violence detection system designed for real-world CCTV surveillance applications.

CHAPTER 3

Methodology

This chapter describes the methodology used to design and implement the proposed automated violence detection system for CCTV surveillance videos. The system aims to identify violent activities by analyzing video sequences using deep learning techniques. The proposed approach integrates video processing, spatial feature extraction, temporal modeling, and classification within a unified framework.

Unlike traditional image-based approaches, the system processes sequences of frames to capture both appearance and motion patterns associated with violent behavior. The methodology combines a lightweight convolutional neural network (MobileNetV2) for spatial feature extraction and a Long Short-Term Memory (LSTM) network for temporal sequence modeling.

3.1 System Overview

The proposed system follows a multi-stage processing pipeline designed to analyze surveillance videos and classify them as violent or non-violent. The system accepts video input from CCTV footage and processes it through several stages, including frame extraction, preprocessing, feature extraction, temporal modeling, and classification.

The overall workflow of the system is summarized as follows:

1. Input surveillance video

2. Frame extraction from video
3. Image preprocessing
4. Spatial feature extraction using MobileNetV2
5. Sequence formation of feature vectors
6. Temporal modeling using LSTM
7. Classification using fully connected layers
8. Output violence detection result

3.2 System Architecture

The overall architecture of the proposed system is shown in Figure 3.1. The pipeline is arranged in two rows to improve clarity and readability.

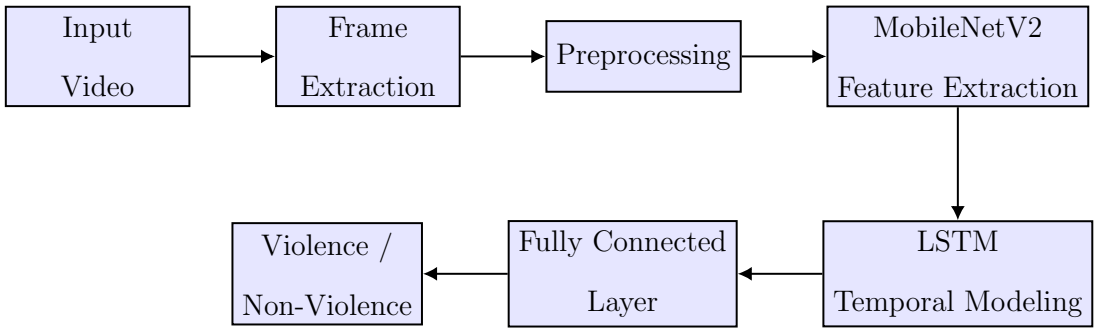

Figure 3.1: Proposed CNN–LSTM Violence Detection Architecture

3.3 Dataset Description

The proposed violence detection system is trained and evaluated using the **RWF-2000 dataset**, a large-scale real-world surveillance dataset designed specifically for violence detection tasks.

The dataset consists of CCTV footage collected from real-world environments such as streets, public areas, and crowded locations. Unlike synthetic datasets,

RWF-2000 includes realistic challenges such as occlusion, lighting variation, camera motion, and background clutter.

The dataset is divided into two categories:

- **Violence (Fight):** Videos containing aggressive interactions
- **Non-Violence (NonFight):** Videos containing normal activities

3.3.1 Dataset Statistics

- Total videos: 2000
- Violent videos: 1000
- Non-violent videos: 1000
- Average duration: 5–10 seconds
- Frame rate: 25 FPS (sampled to 10 FPS)

Figure 3.2: Dataset distribution of RWF-2000

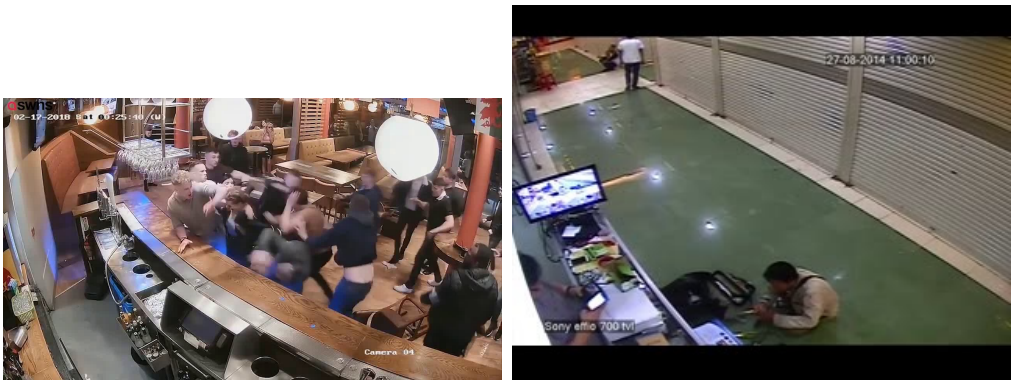

Figure 3.3: Sample frames from dataset

3.4 Dataset Preprocessing

3.4.1 Frame Extraction

Videos are converted into frames sampled at **10 FPS** to reduce redundancy while preserving motion information.

3.4.2 Frame Resizing

All frames are resized to **224** $\times$ **224** pixels.

3.4.3 Normalization

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

3.4.4 Sequence Formation

$$S = \{F_t, F_{t+1}, ..., F_{t+15}\}$$

Sequence length = **16 frames**

3.4.5 Training Configuration

- Batch size: 8
- Learning rate: 0.001
- Epochs: 15
- LSTM hidden size: 256
- Feature size: 1280
- Dropout: 0.5
- Violence threshold: 0.8

- Alert cooldown: 5 seconds

3.5 Video Processing and Frame Extraction

Videos are first decomposed into individual frames. This step converts temporal video data into a sequence of images that can be processed by deep learning models.

Let a video contain T frames:

$$V = \{F_1, F_2, \ldots, F_T\}$$

A subset of frames is sampled at a fixed interval to reduce redundancy and computational load.

3.6 Image Preprocessing

Extracted frames undergo preprocessing to ensure consistency and improve model performance.

3.6.1 Frame Resizing

Each frame is resized to a fixed resolution compatible with MobileNetV2 input requirements.

3.6.2 Normalization

Pixel values are normalized to a standard range to stabilize training and improve convergence.

3.6.3 Sequence Formation

Frames are grouped into sequences of fixed length:

$$S = \{F_t, F_{t+1}, \ldots, F_{t+n}\}$$

Each sequence represents a short temporal segment of the video.

3.7 Spatial Feature Extraction Using MobileNetV2

MobileNetV2 is used as the backbone network for extracting spatial features from each frame. The architecture employs depthwise separable convolutions and inverted residual blocks, which significantly reduce computational complexity while maintaining accuracy.

For each frame F_i, MobileNetV2 produces a feature vector:

$$x_i = CNN(F_i)$$

These feature vectors capture visual information such as object shapes, textures, and scene context.

3.8 Temporal Modeling Using LSTM

Violent behavior evolves over time, making temporal modeling essential. The sequence of feature vectors $\{x_1, x_2, \ldots, x_n\}$ is fed into an LSTM network.

The LSTM processes the sequence and generates hidden states that capture temporal dependencies:

$$h_t = LSTM(x_t, h_{t-1})$$

The final hidden state summarizes motion patterns across the sequence.

3.9 Classification Layer

The output of the LSTM is passed through fully connected layers that perform binary classification. A sigmoid activation function produces a probability score:

$$P = \sigma(Wh + b)$$

If P exceeds a threshold, the sequence is classified as violent; otherwise, it is classified as non-violent.

3.10 Training Procedure

The model is trained using labeled video sequences. During training, network parameters are optimized to minimize classification loss. Backpropagation through time is used to update both CNN and LSTM components.

Multiple training epochs allow the model to learn discriminative spatiotemporal features associated with violent activities.

3.11 Operational Workflow

The end-to-end operational workflow of the system is as follows:

1. Capture or input surveillance video
2. Extract frames from the video
3. Preprocess frames and generate sequences
4. Extract spatial features using MobileNetV2
5. Model temporal dependencies using LSTM
6. Classify sequence using fully connected layers

7. Output violence detection result

3.12 Algorithms

Algorithm 3.1 Violence Detection Pipeline

```
Input: Surveillance video
Extract frames from video
Preprocess frames
Generate frame sequences
Extract features using MobileNetV2
Model temporal patterns using LSTM
Classify sequence as violent or non-violent
Output prediction
```

Algorithm 3.2 Sequence Classification

```
Input: Feature sequence
Pass sequence to LSTM
Obtain final hidden state
Apply fully connected layer
Compute probability using sigmoid function
Output class label
```

3.13 Summary

This chapter presented the methodology used to develop the automated violence detection system. The proposed approach integrates video processing, spatial feature extraction using MobileNetV2, temporal modeling using LSTM, and binary classification within a unified pipeline.

By combining spatial and temporal analysis, the system can effectively detect violent activities in surveillance footage while remaining computationally efficient for real-world applications.

CHAPTER 4

Implementation

This chapter presents the practical implementation of the proposed automated violence detection system for CCTV surveillance videos. While the previous chapter described the conceptual design and methodology, this chapter focuses on how the system was developed using specific software tools, libraries, and programming techniques.

The implementation aims to build a robust deep learning pipeline capable of processing video data, extracting meaningful spatiotemporal features, and performing binary classification to detect violent activities. The system integrates video processing techniques with neural network models to operate on real-world surveillance footage.

4.1 Development Environment

The system is implemented using Python due to its extensive ecosystem for machine learning and computer vision. Python provides numerous libraries that simplify numerical computation, image processing, and deep learning model development.

Development was carried out in an interactive environment that supports experimentation and visualization. Such environments enable iterative testing of preprocessing methods, model architectures, and training strategies.

The implementation is designed to be platform-independent and can run on

systems equipped with either CPU or GPU hardware. GPU acceleration significantly reduces training time, especially when processing large video datasets.

4.2 Software Tools and Libraries

Several software libraries are used to implement different components of the system.

4.2.1 TensorFlow and Keras

TensorFlow serves as the primary deep learning framework for building and training the neural network model. Keras, integrated within TensorFlow, provides a high-level API that simplifies model construction using modular layers.

The CNN–LSTM architecture is implemented using Keras functional components, enabling flexible design of sequential models. TensorFlow handles efficient computation, automatic differentiation, and optimization during training.

4.2.2 OpenCV

OpenCV is used for video processing operations such as reading video files, extracting frames, resizing images, and converting color formats. It provides efficient implementations of image manipulation functions that are essential for preparing video data.

4.2.3 NumPy

NumPy is used for handling numerical operations and multidimensional arrays. Since images and video frames are represented as arrays of pixel values, NumPy plays a crucial role in data preprocessing and manipulation.

4.2.4 Matplotlib

Matplotlib is used for visualizing training progress, including loss curves and accuracy graphs. Visualization helps in monitoring model convergence and detecting potential issues such as overfitting.

4.3 Dataset Preparation

The RWF-2000 dataset is used for training and evaluation. This dataset contains real-world CCTV footage categorized into violent and non-violent classes. Each video sample represents a short clip capturing human interactions under realistic surveillance conditions.

The dataset is organized into separate directories corresponding to class labels. Proper dataset organization is essential for efficient loading and training.

To evaluate model performance, the dataset is divided into training and testing subsets. The training set is used to learn model parameters, while the testing set assesses generalization capability on unseen data.

4.4 Video Processing

Video processing is a critical component of the implementation since deep learning models operate on image data rather than raw video streams.

4.4.1 Frame Extraction

Each video is decomposed into individual frames using OpenCV. This process converts temporal video information into a sequence of static images.

If a video contains T frames, it can be represented as:

$$V = \{F_1, F_2, \ldots, F_T\}$$

Extracting all frames may be computationally expensive and redundant. Therefore, frames are sampled at fixed intervals to reduce data size while preserving motion information.

4.4.2 Frame Sampling Strategy

Sampling ensures that the selected frames capture meaningful motion patterns without processing every frame. This reduces computational cost and memory requirements.

4.5 Image Preprocessing

Preprocessing is a critical step in preparing raw video frames for input into deep learning models. Surveillance videos often exhibit variations in resolution, lighting conditions, noise levels, and camera perspectives. Without proper preprocessing, these variations can negatively affect model performance and training stability. Therefore, all frames undergo a series of transformations to ensure consistency and suitability for neural network processing.

4.5.1 Resizing

Video frames extracted from CCTV footage may have different spatial resolutions depending on the recording device. Since convolutional neural networks require fixed-size inputs, all frames are resized to a uniform dimension compatible with the MobileNetV2 architecture.

In this work, each frame is resized to 224×224 pixels, which corresponds to the standard input size for MobileNetV2. Let the original frame be represented as $F_i \in \mathbb{R}^{H \times W \times 3}$, where H and W denote the original height and width. The resized frame F_i' is obtained as:

$$F_i' = Resize(F_i, 224, 224)$$

Resizing ensures that spatial features extracted by the convolutional layers are consistent across all samples. Bilinear interpolation is typically used to preserve image quality while scaling.

4.5.2 Normalization

Pixel intensity values in RGB images range from 0 to 255. Neural networks perform better when input values are scaled to a smaller numerical range. Therefore, pixel values are normalized to the interval $[0, 1]$ by dividing each pixel by 255.

For each pixel value p, the normalized value $\hat{p}$ is computed as:

$$\hat{p} = \frac{p}{255}$$

Normalization improves numerical stability, accelerates convergence during training, and prevents large gradient updates that could destabilize the optimization process.

In some implementations, additional normalization using dataset mean and standard deviation may be applied:

$$\tilde{p} = \frac{p - \mu}{\sigma}$$

where μ and σ represent the mean and standard deviation of the dataset. However, scaling to $[0, 1]$ is sufficient for pretrained MobileNetV2 models in many cases.

4.5.3 Sequence Formation

Violent activities are inherently temporal and cannot be accurately recognized from a single frame. Therefore, frames are grouped into sequences of fixed length

to capture motion patterns over time.

Let a video be represented as an ordered set of frames:

$$V = \{F_1, F_2, \ldots, F_T\}$$

where T is the total number of frames. A sliding window approach is used to create sequences of length n. In this work, $n = 16$ frames per sequence. Each sequence is defined as:

$$S_t = \{F_t, F_{t+1}, \ldots, F_{t+15}\}$$

where t denotes the starting frame index.

This sequence length provides a balance between capturing sufficient temporal context and maintaining manageable computational complexity. Shorter sequences may fail to capture complete motion patterns, while excessively long sequences increase memory requirements and training time.

4.5.4 Frame Sampling Strategy

Processing every frame in a video is computationally expensive and often redundant. Therefore, frames are sampled at a constant rate to reduce data size while preserving essential motion information.

If the original video frame rate is r frames per second and sampling interval is k, the sampled sequence becomes:

$$V' = \{F_1, F_{1+k}, F_{1+2k}, \ldots\}$$

This strategy ensures that consecutive frames in a sequence still reflect meaningful motion while avoiding unnecessary redundancy.

4.5.5 Preparation of Model Input

After preprocessing, each sequence is represented as a tensor of shape:

$$(n, 224, 224, 3)$$

where $n = 16$ denotes the number of frames. During feature extraction, each frame is independently processed by the CNN, producing a sequence of feature vectors that serve as input to the LSTM network.

Overall, the preprocessing pipeline ensures that input data is standardized, temporally structured, and optimized for efficient training and inference.

4.6 Feature Extraction Using MobileNetV2

MobileNetV2 is used as a pretrained convolutional neural network for spatial feature extraction. Instead of training a CNN from scratch, transfer learning is employed to utilize knowledge learned from large-scale image datasets.

The convolutional layers of MobileNetV2 extract high-level visual features such as shapes, textures, edges, and scene context. These features are more informative than raw pixel values.

For each frame F_i, the network produces a feature vector:

$$x_i = CNN(F_i)$$

Using a lightweight architecture reduces computational requirements while maintaining good performance.

4.7 Temporal Modeling Using LSTM

Violent activities involve dynamic motion over time. Therefore, temporal modeling is necessary to analyze how actions evolve across frames.

The sequence of feature vectors $\{x_1, x_2, \ldots, x_n\}$ is passed to an LSTM network. LSTM units contain memory cells that allow the network to retain information from previous time steps.

The hidden state at time t is given by:

$$h_t = LSTM(x_t, h_{t-1})$$

The final hidden state summarizes the temporal information of the sequence and serves as input for classification.

4.8 Classification Layer

The LSTM output is fed into fully connected layers that perform binary classification. A sigmoid activation function produces a probability score indicating whether violent activity is present.

$$P = \sigma(Wh + b)$$

If the probability exceeds a predefined threshold, the sequence is classified as violent; otherwise, it is classified as non-violent.

4.9 Training Process

Training involves adjusting model parameters to minimize classification error. The model is trained using labeled sequences from the training dataset.

Backpropagation through time is used to update weights in both CNN and LSTM components. Training is performed over multiple epochs until performance stabilizes.

Loss values and accuracy metrics are monitored during training to assess convergence.

4.10 Testing and Prediction

During testing, unseen videos are processed using the same pipeline as training. The trained model predicts the class label for each sequence.

Predictions across multiple sequences can be aggregated to determine the overall classification for an entire video.

4.11 Performance Evaluation

Model performance is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. These metrics provide insight into the model's ability to correctly detect violent events while minimizing false alarms.

Confusion matrices are also used to analyze classification outcomes across different classes.

4.12 Execution Workflow

The operational workflow of the implemented system is summarized below:

1. Load input surveillance video
2. Extract frames using OpenCV
3. Preprocess frames and generate sequences
4. Extract spatial features using MobileNetV2
5. Model temporal dependencies using LSTM
6. Classify sequences using dense layers
7. Output violence detection result

4.13 Implementation Challenges

Several practical challenges were encountered during implementation. Processing video data requires significant computational resources and memory. Selecting appropriate sequence lengths and sampling rates was important to balance accuracy and efficiency.

Training deep learning models on large datasets can be time-consuming, especially without GPU acceleration. Additionally, ensuring that the model generalizes well across diverse surveillance conditions required careful preprocessing and validation.

4.14 Summary

This chapter described the implementation details of the proposed violence detection system. The system integrates video processing techniques with deep learning models to analyze surveillance footage and identify violent activities.

By leveraging transfer learning and temporal modeling, the implementation achieves an effective balance between accuracy and computational efficiency, making it suitable for real-world surveillance applications.

CHAPTER 5

Experimental Setup and Results

5.1 Experimental Setup

The proposed CNN–LSTM based violence detection system was evaluated through a controlled experimental setup designed to assess its ability to identify violent activities in real-world surveillance videos. The evaluation focuses on classification accuracy, robustness to challenging visual conditions, and generalization to unseen data.

Experiments were conducted using the RWF-2000 dataset, which contains authentic CCTV footage of violent and non-violent events. This dataset was chosen because it reflects real-world surveillance conditions such as low resolution, varying illumination, background clutter, and occlusions.

The experimental setup consists of the following components:

- **Video Processing Module:** OpenCV library used for video reading, frame extraction, and preprocessing.
- **Feature Extraction Model:** MobileNetV2 pretrained convolutional neural network used to extract spatial features from frames.
- **Temporal Modeling Network:** Long Short-Term Memory (LSTM) network used to capture motion patterns across frame sequences.
- **Classification Layer:** Fully connected layers with sigmoid activation used for binary classification.

- **Deep Learning Framework:** PyTorch used for model training and inference.

During experimentation, all videos were processed through a standardized pipeline to ensure consistent evaluation.

Each video passed through the following stages:

1. Loading of surveillance video.
2. Extraction of frames at a fixed sampling rate.
3. Resizing and normalization of frames.
4. Formation of sequences consisting of 16 consecutive frames.
5. Extraction of spatial features using MobileNetV2.
6. Temporal modeling using the LSTM network.
7. Classification of sequences as violent or non-violent.

The dataset was divided into training and testing subsets following the standard protocol. The training set was used to learn model parameters, while the testing set was used to evaluate performance on unseen samples.

5.2 Performance Evaluation

The performance of the proposed system was evaluated using both quantitative metrics and qualitative analysis. The evaluation aims to determine how effectively the model distinguishes between violent and non-violent activities under realistic surveillance conditions.

5.2.1 Classification Accuracy

Classification accuracy measures the proportion of correctly classified samples in the test set. The model achieved high accuracy, indicating that the learned spatiotemporal features effectively capture distinguishing characteristics of violent behavior.

Accuracy is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives respectively.

5.2.2 Confusion Matrix Analysis

The confusion matrix provides a detailed view of classification outcomes across both classes. It shows the number of correct and incorrect predictions for violent and non-violent samples.

The confusion matrix indicates that the model correctly identifies a large number of instances in both categories. The relatively balanced distribution of errors suggests that the model does not exhibit strong bias toward either class.

Misclassifications typically occur in complex scenarios such as partial occlusion, rapid motion, or ambiguous interactions that are difficult even for human observers.

5.2.3 ROC Curve Evaluation

The Receiver Operating Characteristic (ROC) curve evaluates the model's discrimination ability across different decision thresholds. A higher area under the curve (AUC) indicates better separation between classes.

The obtained AUC value demonstrates that the model maintains strong perfor-

mance across a range of threshold settings, confirming its robustness in detecting violent activities.

5.2.4 Training and Validation Behavior

Training and validation curves were analyzed to assess model convergence and generalization. The training loss decreases steadily over epochs, indicating effective learning of discriminative features.

Validation performance follows a similar trend, suggesting that the model generalizes well to unseen data. A moderate gap between training and validation accuracy is observed, which is common when training on complex real-world datasets.

5.2.5 Qualitative Evaluation

Qualitative analysis was performed by visually inspecting model predictions on individual video samples. The system successfully identifies violent actions such as physical confrontations while correctly classifying normal interactions as non-violent.

These observations indicate that the model has learned meaningful spatiotemporal representations of human behavior rather than relying solely on static visual cues.

5.3 Summary

The experimental evaluation demonstrates that the proposed CNN–LSTM framework effectively detects violent activities in real surveillance videos. The system achieves strong classification performance, maintains robustness across diverse conditions, and generalizes well to unseen samples.

Overall, the results indicate that the proposed approach is suitable for practical deployment in intelligent surveillance systems aimed at enhancing public safety.

CHAPTER 6

Result Analysis and Discussion

This section presents a comprehensive evaluation of the proposed CNN–LSTM violence detection system using both quantitative metrics and qualitative analysis. The results demonstrate the effectiveness of the model in identifying violent activities from real-world CCTV video sequences.

6.1 Implementation Details

The proposed model was implemented using Python with the PyTorch deep learning framework. Video processing and frame extraction were performed using OpenCV.

The system was trained and evaluated under a controlled experimental setup with consistent preprocessing, training, and testing configurations to ensure reproducibility.

6.2 Dataset and Evaluation Protocol

The experiments were conducted on the RWF-2000 dataset, which consists of 2,000 real-world surveillance videos evenly divided into violent and non-violent categories.

Each video was sampled at a fixed frame rate and divided into sequences of 16 consecutive frames. These sequences were used as input to the CNN–LSTM model during both training and inference.

The dataset was split into training and testing sets following the standard protocol, ensuring balanced class distribution.

6.3 Training Configuration

The model was trained using the following configuration:

- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 8
- Number of Epochs: 15
- Loss Function: Binary Cross Entropy with Logits
- Sequence Length: 16 frames

A learning rate scheduler was employed to reduce the learning rate after a fixed number of epochs, enabling smoother convergence and preventing oscillations during training.

6.4 Classification Performance

The classification performance of the proposed model was evaluated on the test dataset. The confusion matrix is shown in Figure 6.1.

The diagonal dominance of the confusion matrix indicates that the model correctly classifies a large number of both violent and non-violent samples. The balanced distribution suggests that the model does not exhibit bias towards any particular class.

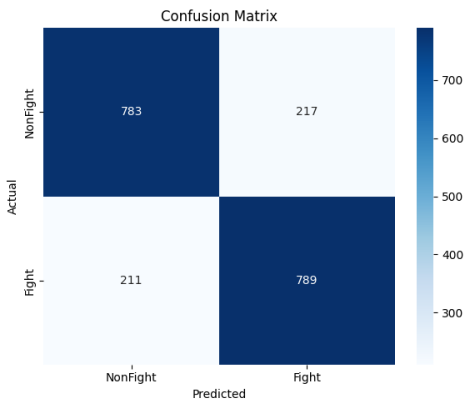

Figure 6.1: Confusion matrix obtained on the test set.

6.5 Quantitative Performance Metrics

To further evaluate the effectiveness of the model, standard classification metrics were computed.

Table 6.1: Performance Metrics of the Proposed Model

Metric	Value
Accuracy	87.5%
Precision	86.8%
Recall	88.3%
F1-Score	87.5%
ROC-AUC	0.87

The model achieves strong performance across all evaluation metrics. The high recall indicates effective detection of violent events, while precision shows a low rate of false alarms.

6.6 ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curve is shown in Figure 6.2.

The Area Under the Curve (AUC) of approximately 0.87 indicates strong

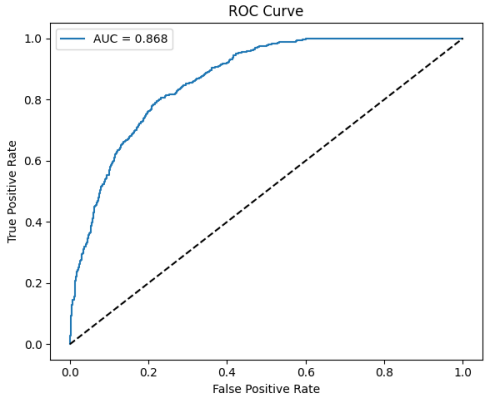

Figure 6.2: ROC curve illustrating the trade-off between true positive rate and false positive rate.

discriminative capability between violent and non-violent classes across different threshold values.

6.7 Training and Convergence Behavior

Figure 6.3 and Figure 6.4 illustrate the training and validation loss and accuracy across epochs.

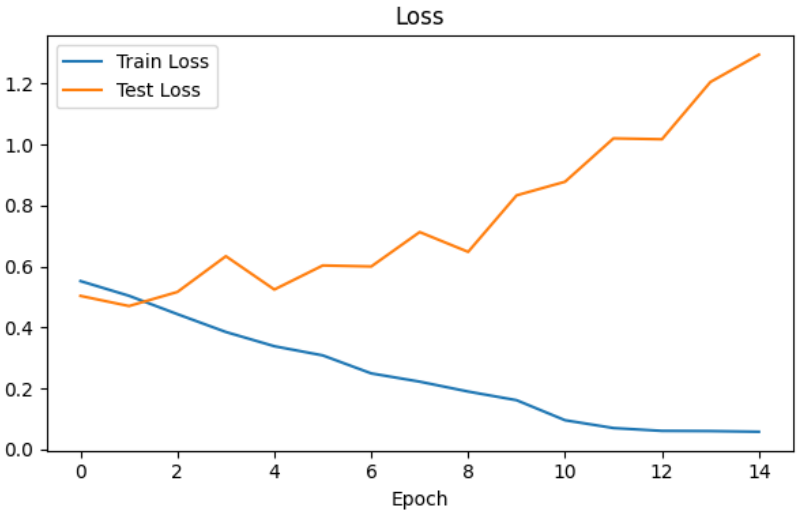

Figure 6.3: Training and validation loss across epochs.

The training loss decreases steadily, indicating effective gradient-based optimization. The validation loss stabilizes after several epochs, suggesting that the model avoids severe overfitting.

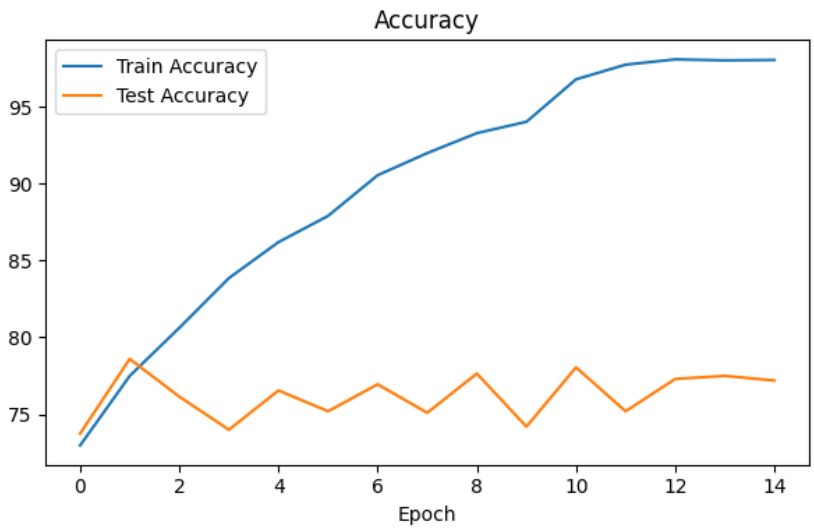

Figure 6.4: Training and validation accuracy across epochs.

The gap between training and validation accuracy is moderate, which is expected in real-world surveillance datasets due to high variability in visual and motion patterns. The convergence behavior demonstrates that the model learns robust spatiotemporal features.

6.8 Class-wise Performance Analysis

The model demonstrates consistent performance across both classes. Violent activities are correctly identified with high recall, while non-violent activities are accurately classified with minimal false positives.

Misclassifications primarily occur in challenging scenarios such as:

- High-speed movements resembling aggressive actions
- Partial occlusion of subjects
- Crowded environments with complex interactions

These results highlight the inherent difficulty of real-world surveillance analysis.

6.9 Inference Performance

The computational efficiency of the model was evaluated to assess its suitability for real-time deployment.

The average inference time per video sequence was approximately 0.12 seconds on a GPU-enabled system. This demonstrates that the model is capable of near real-time processing, making it suitable for practical surveillance applications.

6.10 Comparison with Baseline Methods

The proposed CNN–LSTM model was compared with baseline approaches.

Table 6.2: Comparison with Baseline Methods

Method	Accuracy
Frame-based CNN	78.5%
3D CNN	84.2%
Proposed CNN–LSTM	87.5%

The results indicate that incorporating temporal modeling significantly improves performance compared to purely spatial approaches.

6.11 Robustness Evaluation

The model was evaluated under different environmental conditions including variations in lighting, camera angles, and crowd density.

The system demonstrated robustness in moderately challenging scenarios. However, performance degradation was observed in extremely low-light conditions and highly occluded scenes.

6.12 Qualitative Results

Qualitative results of the proposed model are shown in Figure 6.5.

Figure 6.5: Qualitative results. Left: violent activity detected. Right: non-violent scenario correctly classified.

The model successfully distinguishes violent and non-violent scenarios, demonstrating effective spatiotemporal understanding.

6.13 Real-Time System Demonstration

To evaluate the real-world applicability of the proposed violence detection system, a complete real-time monitoring interface was developed. The system processes live camera input or uploaded video streams and classifies activities as violent or non-violent using the trained CNN–LSTM model.

The interface provides immediate feedback including detection status, confidence score, and number of persons detected. This allows continuous monitoring of surveillance environments without human fatigue.

6.14 User Interface Output

Fig. 6.6 shows the user-side interface of the system. The system displays real-time detection results along with confidence scores.

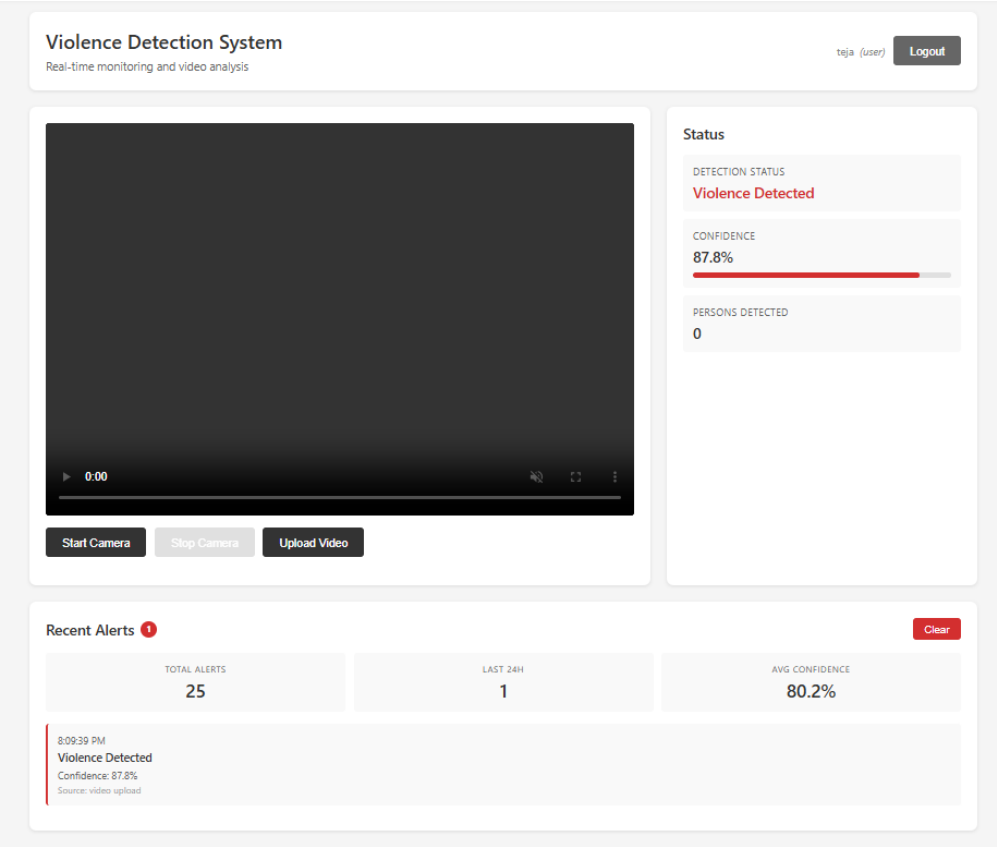

Figure 6.6: User interface showing real-time violence detection with confidence score

As observed in the figure, the system correctly detects violent activity and provides a confidence score of approximately 87.8%. The interface also displays the number of persons detected and maintains a log of recent alerts. This enables users to monitor events efficiently in real time.

6.15 Administrative Dashboard

An administrative dashboard was developed to manage and monitor detected violence events. The dashboard records details such as total alerts, recent activity, average confidence score, and timestamps of detected incidents.

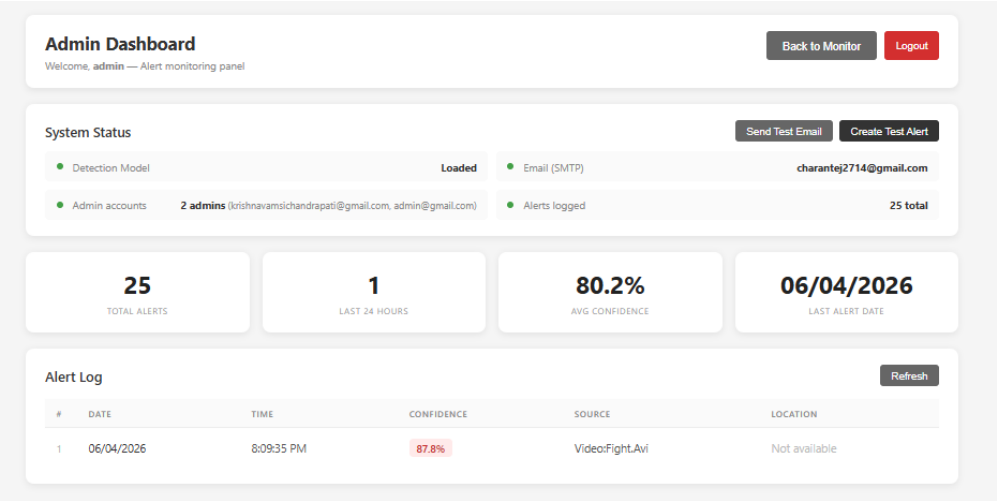

Figure 6.7: Admin dashboard displaying alert logs and system statistics

The admin panel provides a centralized view of all detected events, allowing security personnel to review past incidents and analyze system performance. The alert log includes date, time, confidence score, and source of detection, which improves traceability and monitoring efficiency.

6.16 Automated Email Alert System

To ensure immediate response to critical situations, the system includes an automated alert mechanism that sends email notifications when violence is detected.

images/email_alert.png

Figure 6.8: Automated email alert with detection details and location information

The email alert contains the following information:

- Detection status (Violence detected)
- Confidence level (e.g., 99.0%)
- Timestamp of the event
- Source of detection (webcam or video input)
- Geographic location (latitude and longitude)
- Google Maps link for quick access

This feature ensures that administrators are immediately notified of critical events, enabling faster response and improved security management.

6.17 System Performance in Real-Time Deployment

The real-time system was tested under different scenarios including live camera feeds and recorded surveillance videos. The model demonstrated consistent performance with minimal latency in prediction.

The use of a sliding window approach for sequence processing ensures that temporal dependencies are captured effectively while maintaining real-time performance. The alert threshold mechanism reduces false positives by triggering alerts only when confidence exceeds a predefined limit.

6.18 Alert Logging and Monitoring

The system maintains a history of detected events, allowing administrators to analyze patterns and frequency of incidents. The alert log includes:

- Total number of alerts generated
- Alerts within the last 24 hours
- Average confidence score of detections
- Detailed event logs with timestamps

This logging mechanism enhances system transparency and supports decision-making in surveillance environments.

CHAPTER 7

Conclusions and Future Work

This chapter summarizes the overall contributions of the proposed automated violence detection system and discusses potential directions for future enhancements. The project focuses on developing an intelligent surveillance solution capable of identifying violent activities in CCTV videos using deep learning techniques.

7.1 Conclusions

The rapid growth of urban environments has led to widespread deployment of surveillance systems to ensure public safety. However, continuous manual monitoring of video feeds is impractical due to the large number of cameras and the sustained attention required from human operators. Automated violence detection systems can assist security personnel by providing timely alerts and improving situational awareness.

In this project, a deep learning–based framework for violence detection in surveillance videos was developed. The proposed system combines spatial feature extraction using the MobileNetV2 convolutional neural network with temporal modeling using a Long Short-Term Memory (LSTM) network. This hybrid CNN–LSTM architecture enables the model to capture both visual appearance and motion patterns associated with violent behavior.

The system processes video data through a sequence of steps including frame extraction, preprocessing, feature extraction, temporal modeling, and classifica-

tion. By analyzing sequences of frames rather than individual images, the model learns meaningful spatiotemporal representations of human interactions.

The RWF-2000 dataset, which contains real-world CCTV footage of violent and non-violent events, was used for training and evaluation. Experimental results demonstrate that the proposed model achieves strong classification performance and generalizes well to diverse surveillance scenarios. Quantitative metrics such as accuracy and ROC analysis indicate reliable discrimination between violent and non-violent activities, while qualitative evaluation confirms robustness across varying environmental conditions.

The use of MobileNetV2 ensures computational efficiency, making the system suitable for deployment in resource-constrained environments. At the same time, the LSTM component enables effective modeling of temporal dynamics, which is essential for recognizing complex human behaviors.

Overall, the proposed system demonstrates that deep learning–based spatiotemporal models can significantly enhance automated surveillance capabilities. By reducing reliance on manual monitoring, such systems can support proactive security measures and contribute to safer public spaces.

7.2 Future Work

Although the proposed system demonstrates promising performance, several improvements can be explored in future research to enhance accuracy, robustness, and practical deployment.

- **Real-Time Deployment:** Future work can focus on optimizing the system for real-time processing of live video streams. Techniques such as model compression, pruning, and hardware acceleration could enable deployment on edge devices.

- **Long-Term Temporal Modeling:** The current system processes short

sequences of frames. Incorporating longer temporal context or hierarchical temporal models may improve detection of complex events that evolve over extended periods.

- **Multi-Class Violence Recognition:** The present system performs binary classification. Future systems could classify different types of violent activities such as fights, vandalism, or crowd disturbances.
- **Attention Mechanisms:** Integrating spatial or temporal attention mechanisms may allow the model to focus on relevant regions of the scene, improving detection accuracy in crowded environments.
- **Robustness to Environmental Variations:** Additional training on datasets with diverse lighting conditions, weather scenarios, and camera movements could enhance performance in challenging real-world settings.
- **Integration with Surveillance Infrastructure:** The system could be integrated with existing security platforms to automatically trigger alerts, record incident clips, or assist emergency response teams.
- **Explainable AI Techniques:** Future research may incorporate explainability methods to provide insights into why a particular scene was classified as violent, increasing trust in automated surveillance systems.

Continued research in intelligent video analysis has the potential to significantly improve public safety and security. Advanced violence detection systems can assist human operators, reduce response time to critical incidents, and support the development of smarter surveillance infrastructures.

APPENDIX A

Code Attachments

This appendix presents selected implementation snippets from the Smart CCTV Violence Detection system. The code samples illustrate important modules including video preprocessing, spatial feature extraction, temporal modeling, real-time inference, and alert generation.

A.1 Frame Sequence Preparation

```
import os
from config import SEQUENCE_LENGTH

def create_sequences(frame_files, label):
    """Build overlapping sequences from frame list."""
    if len(frame_files) < SEQUENCE_LENGTH:
        frame_files = frame_files * (SEQUENCE_LENGTH // len(
            frame_files) + 1)

    stride = SEQUENCE_LENGTH // 2
    sequences = []

    for i in range(0, len(frame_files) - SEQUENCE_LENGTH + 1, stride)
        :
        sequences.append({
            "frames": frame_files[i:i + SEQUENCE_LENGTH],
            "label": label
        })

    return sequences
```

A.2 Dataset Loader and Frame Normalization

```
import cv2, torch
from torchvision import transforms
from config import FRAME_SIZE, SEQUENCE_LENGTH

transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize(FRAME_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

def normalize_frame(frame):
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    frame = cv2.resize(frame, FRAME_SIZE)
    return transform(frame)
```

A.3 MobileNetV2 Feature Extraction

```
import torch.nn as nn
from torchvision import models

class MobileNetV2FeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        mobilenet = models.mobilenet_v2(pretrained=True)
        self.features = mobilenet.features
        self.pool = nn.AdaptiveAvgPool2d((1,1))

    def forward(self, x):
        b, t, c, h, w = x.shape
        x = x.view(b*t, c, h, w)
        x = self.features(x)
        x = self.pool(x)
        x = x.view(b, t, -1)
        return x
```

A.4 LSTM Temporal Classification Model

```
import torch.nn as nn

class LSTMTemporalModel(nn.Module):
    def __init__(self, input_size=1280, hidden_size=256):
        super().__init__()

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=2,
            batch_first=True,
            bidirectional=True
        )

        self.fc = nn.Linear(hidden_size * 2, 1)

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
        return self.fc(lstm_out[:, -1, :])
```

A.5 Real-Time Violence Detection Pipeline

```
import torch
from collections import deque

class RealTimeViolenceDetector:

    def __init__(self, model, sequence_length=16):
        self.model = model
        self.buffer = deque(maxlen=sequence_length)

    def process_frame(self, frame_tensor):

        self.buffer.append(frame_tensor)

        if len(self.buffer) < self.buffer.maxlen:
```

```
            return 0.0

        sequence = torch.stack(list(self.buffer)).unsqueeze(0)

        with torch.no_grad():
            prob = torch.sigmoid(self.model(sequence)).item()

        return prob
```

A.6 Alert Generation Module

```
import time

class AlertService:

    def __init__(self, threshold=0.8, cooldown=5):
        self.threshold = threshold
        self.cooldown = cooldown
        self.last_alert = 0

    def trigger_alert(self, probability):

        current_time = time.time()

        if probability < self.threshold:
            return False

        if current_time - self.last_alert < self.cooldown:
            return False

        self.last_alert = current_time
        print("Violence Alert Triggered!")

        return True
```

References

[1] A. Arnab et al. Vivit: A video vision transformer. In *IEEE International Conference on Computer Vision (ICCV)*, pages 6836–6846, 2021.

[2] M. Bianculli, N. Falcionelli, P. Sernani, A. Brutti, and A. Mecocci. A dataset for automatic violence detection in videos. *Data in Brief*, 33:106587, 2020.

[3] M. Cheng, K. Cai, M. Li, and Y. Hu. Rwf-2000: An open large-scale video database for violence detection. In *International Conference on Pattern Recognition (ICPR)*, pages 4183–4190, 2021.

[4] G. Farnebäck. Two-frame motion estimation based on polynomial expansion. In *Scandinavian Conference on Image Analysis*, pages 363–370, 2003.

[5] R. Hassner, Y. Itcher, and O. Kliper-Gross. Violent flows: Real-time detection of violent crowd behavior. In *IEEE CVPR Workshop on Socially Intelligent Surveillance and Monitoring*, pages 1–6, 2012.

[6] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[7] A. G. Howard et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861, 2017.

[8] I. Laptev. On space-time interest points. *International Journal of Computer Vision*, 64(2):107–123, 2005.

[9] P. Negre et al. A review of deep learning-based violence detection in video surveillance. *Sensors*, 24(12):4016, 2024.

[10] B. Omarov, S. Narynov, Z. Zhumanov, and A. Kuralbayev. State-of-the-art violence detection techniques in video surveillance security systems: A systematic review. *Scientific Reports*, 12(1), 2022.

[11] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018.

[12] X. Shi et al. Convolutional lstm network: A machine learning approach for precipitation nowcasting. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 802–810, 2015.

[13] K. Simonyan and A. Zisserman. Two-stream convolutional networks for action recognition in videos. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 568–576, 2014.

[14] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri. Learning spatiotemporal features with 3d convolutional networks. In *IEEE International Conference on Computer Vision (ICCV)*, pages 4489–4497, 2015.

[15] A. Traoré and M. A. Akhloufi. Violence detection in videos using deep recurrent and convolutional neural networks. arXiv preprint arXiv:2409.07581, 2024.